



**EUROPEAN COMMISSION
DIRECTORATE-GENERAL FOR COMMUNICATIONS NETWORKS, CONTENT AND
TECHNOLOGY**

Artificial Intelligence Office
Artificial Intelligence for Societal Good

**LDT_Toolbox_D03_06_EU_LDT_Installation_Manual_
EU_Building_Database**

**CNECT/2023/OP/0051 - Development of the Local Digital Twins (LDT)
Toolbox for Advancing the Transformation of Smart Communities**

This document has been prepared for the Local Digital Twins Toolbox project. The project is an initiative of the European Commission. Directorate-General for Communications Networks, Content and Technology of the European Union is responsible for contract management of the Local Digital Twins Toolbox project.

European Commission

Directorate-General for Communications Networks, Content and Technology
Unit A.5 Artificial Intelligence for Societal Good

Last update: 17 08 2026

Task	Task 3 Development of EU LDT Toolbox
Deliverable	LDT_Toolbox_D03_06_EU_LDT_Installation_Manual_EU_Building_Database
Authors:	EU LDT Toolbox Consortium
Reviewers:	Svetlogor Kirov

VERSIONING AND CHANGE LOG TABLE

Version	Date	Author	Changes	Affected Sections
1	10-08-2026	Laurens J.N. Oostwegel	Draft version	All
2	11-08-2026	Pablo de la Mora	Reviewed version	All
3	13-08-2026	Danijel Schorlemmer	Reviewed version	All

TABLE OF CONTENTS

1	INTRODUCTION	5
1.1	DOCUMENT OVERVIEW.....	5
1.2	PURPOSE OF THIS DOCUMENT	5
1.3	TARGET AUDIENCE	5
2	POSTGRESQL WITH POSTGIS INSTALLATION	6
2.1	POSTGRESQL INSTALLATION	6
2.2	POSTGIS INSTALLATION	6
2.3	CREATE DATABASE AND USERS	6
2.4	ENABLE POSTGIS EXTENSION	7
2.5	IMPORT DUMP FILES	7
2.6	ACCESS DATABASE WITH QGIS	8
3	DATABASE STRUCTURE	10
3.1	HOW THE TABLES RELATE	10
3.2	UNDERSTANDING NUMBER AND RELATIVE_SIZE.....	11
4	SUPPORT AND CONTACT INFORMATION	12
4.1	ADDITIONAL RESOURCES	12
5	FAQS	13
6	GLOSSARY	14

LIST OF FIGURES

FIGURE 1. HOW THE ENTITIES, ENTITY_ASSETS, ASSETS, TAXONOMIES AND SOURCES TABLES RELATE..... 10

1 Introduction

1.1 Document overview

This document describes the installation of a PostgreSQL database with the PostGIS spatial extension used to host the EU Building Database exposure data within the Local Digital Twins (LDT) Toolbox. It covers the installation of PostgreSQL and PostGIS on Ubuntu LTS 24.04, the creation of the database and its user roles, the import of the provided schema and data dump files, and the configuration of a connection from QGIS to explore the imported layers.

The same steps apply to other platforms, such as Windows, macOS, or other Linux distributions; only the commands used to install the packages differ.

A separate manual describing how to explore and use the exposure data once it has been imported is available at: https://docs.dynamicexposure.org/gde/getting_started/ExploreData/.

This document is the counterpart to the User Manual, which covers connecting to and querying the database once it is running; the two are meant to be used together.

1.2 Purpose of this document

The manual is divided into the following key sections:

- **[PostgreSQL Installation](#)**: setting up the PostgreSQL repository and installing PostgreSQL 17 on Ubuntu LTS 24.04.
- **[PostGIS Installation](#)**: installing the PostGIS spatial extension for PostgreSQL.
- **[Database and user setup](#)**: creating the exposure database and the consumer, contributor, read_only, and read_write roles.
- **[Import of dump files](#)**: importing the provided schema and data SQL dump files into the database.
- **[Access with QGIS](#)**: connecting to the database from QGIS to load and explore the exposure layers.

Each chapter can also be used as a standalone reference once the initial installation is complete.

1.3 Target audience

This document is intended for technical users who need to set up a local or server instance of the EU Building Database, including researchers and developers requiring direct database access, and GIS users who wish to connect to the database with QGIS. Basic familiarity with the Linux command line and PostgreSQL is recommended.

No prior experience with PostGIS or SpatiaLite is assumed; each step below includes the exact commands to run.

2 PostgreSQL with PostGIS Installation

This chapter describes the installation of PostgreSQL with PostGIS on Ubuntu LTS 24.04, the creation of a database, and the import of schema and data dump files. This setup also works on other platforms (Windows, macOS, other Linux distributions); the commands used to install the packages will be different.

Where a command depends on your specific setup, such as a password or a file path, it is shown below as a placeholder to be replaced before running it.

2.1 PostgreSQL Installation

2.1.1 Set up repository

```
sudo apt install -y postgresql-common  
sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh  
sudo apt update
```

For other platforms, see the [PostgreSQL documentation](#).

This adds the official PostgreSQL Global Development Group (PGDG) repository, which provides current PostgreSQL and PostGIS packages for Ubuntu ahead of what the default Ubuntu repositories carry.

2.1.2 Install PostgreSQL

This setup should also work with PostgreSQL version 18, but version 17 is used here.

```
sudo apt install -y postgresql-17 postgresql-client-17
```

- `postgresql-17` installs the database server itself; `postgresql-client-17` installs `psql`, the command-line client used throughout the rest of this chapter.

2.2 PostGIS Installation

```
sudo apt install -y postgis postgresql-17-postgis-3
```

- `postgresql-17-postgis-3` must match the installed PostgreSQL version (17 here); installing a PostGIS package built for a different server version will not load.

2.3 Create database and users

```
sudo -u postgres createdb -E UTF8 exposure  
sudo -u postgres psql
```

In the `psql` prompt:

```
CREATE ROLE consumer WITH LOGIN PASSWORD 'password_for_consumer';  
CREATE ROLE contributor WITH LOGIN PASSWORD 'password_for_contributor';  
CREATE ROLE read_only;  
CREATE ROLE read_write;  
GRANT read_only TO consumer;
```

```
GRANT read_write TO contributor;  
GRANT CONNECT ON DATABASE exposure TO read_only;  
GRANT CONNECT ON DATABASE exposure TO read_write;  
GRANT USAGE ON SCHEMA public TO read_only, read_write;  
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO  
read_only;  
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT, INSERT, UPDATE,  
DELETE, TRUNCATE ON TABLES TO read_write;  
¥q
```

`consumer` and `contributor` are the roles applications and users actually log in with; `read_only` and `read_write` are only group privileges together, following the general PostgreSQL practice of granting privileges to group roles rather than to login roles directly.

2.4 Enable PostGIS extension

```
sudo -u postgres psql -d exposure
```

In the psql prompt:

```
CREATE EXTENSION postgis;  
¥q
```

PostGIS is enabled per database rather than server-wide, so this step is needed once for the exposure database even if PostGIS was already used elsewhere on the same server.

2.5 Import dump files

The schema dump and data dump are typically provided together as part of an EU Building Database release; both are needed; the schema dump on its own creates empty tables.

```
sudo -u postgres psql -d exposure -f /path/to/schema.sql
```

This must run before the data dump, since it creates the tables that the data dump's records are inserted into.

2.5.1 Import data dump

```
sudo -u postgres psql -d exposure -f /path/to/data.sql
```

Depending on the size of the release (a single country versus the full EU), this step can take anywhere from a few seconds to a considerable amount of time.

2.5.2 Set permissions for imported tables

```
sudo -u postgres psql -d exposure
```

In the psql prompt:

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO read_only;  
GRANT SELECT, INSERT, UPDATE, DELETE, TRUNCATE ON ALL TABLES IN SCHEMA  
public TO read_write;  
GRANT USAGE ON SCHEMA exposure TO read_only, read_write;  
GRANT SELECT ON ALL TABLES IN SCHEMA exposure TO read_only;  
GRANT SELECT, INSERT, UPDATE, DELETE, TRUNCATE ON ALL TABLES IN SCHEMA  
exposure TO read_write;
```

This step is needed because ALTER DEFAULT PRIVILEGES (used earlier) only applies to objects created after it was run; it does not retroactively cover the tables just created by the schema dump.

2.6 Access database with QGIS

For this to work, the database port must be accessible from the QGIS workstation.

If the QGIS workstation is not on the same machine as the database, this typically means opening port 5432 in the server's firewall for that workstation's IP address specifically, rather than to the internet at large.

2.6.1 Add PostgreSQL connection

To add a PostgreSQL connection:

1. Open QGIS.
2. Go to Layer.
3. Select 'Add Layer'.
4. Add 'PostGIS Layers'.
5. Click 'New' to create a new connection.
6. Enter connection details:
 - **Name:** exposure (or any descriptive name)
 - **Host:** localhost (or the server IP address)
 - **Port:** 5432
 - **Database:** exposure
 - **Username:** consumer (read-only) or contributor (read-write)
 - **Password:** enter the password for the selected user
7. Click 'Test Connection' to verify.
8. Click 'OK'.

This connection is saved in QGIS and can be reused for future sessions without re-entering the details.

2.6.2 Load layers

To load layers:

1. Select the connection in the 'Add PostGIS Layers' dialogue.
2. Choose the schema (exposure).
3. Select the tables/layers you want to add.
4. Click Add to load them into the map.

Multiple tables can be selected at once in this dialogue, so the entities, entity_assets and assets tables (used together in the database structure) can be added in a single step.

3 Database structure

Now that the database has been installed and populated, this chapter explains how its data is structured.

The same structure applies to both the PostGIS and SpatiaLite versions of the database; only the schema qualification differs (exposure.entities in PostGIS versus entities in SpatiaLite).

3.1 How the tables relate

The exposure model is built around three core tables: `entities`, `entity_assets` and `assets`.

- `entities` describes the geographical features of the model, either a tile (a Quadtree cell aggregating many buildings) or an individual building.
- `assets` describes a possible building type (its taxonomy and attributes such as population and cost), independently of any specific location.
- `entity_assets` is the junction table connecting the two: it links an entity to each asset type that could apply to it, weighted by the number field.

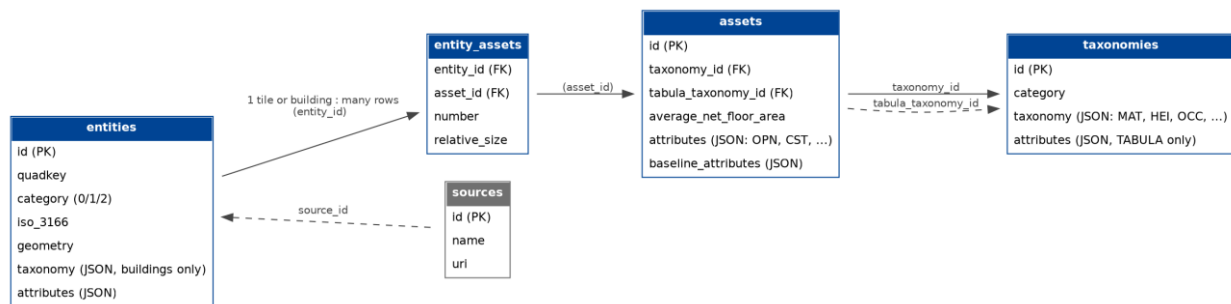


FIGURE 1. HOW THE ENTITIES, ENTITY_ASSETS, ASSETS, TAXONOMIES AND SOURCES TABLES RELATE.

- **entities, entity_assets and assets:** `entity_assets.entity_id` matches `entities.id`, and `entity_assets.asset_id` matches `assets.id`.
- **assets.taxonomy_id / assets.tabula_taxonomy_id:** both reference the `taxonomies` table.
- **entities.source_id:** references the `sources` table, recording where a building's geometry originates from (e.g. OpenStreetMap).
- **district_assets:** mirrors `entity_assets`, but aggregates assets by administrative boundary (a district or country) rather than by individual entity.
- **metadata:** a general-purpose key/value table used for recording metadata about the database itself.

Entities are indexed by quadkey and by category, so looking up a tile by Quadkey or filtering tiles from buildings is fast even on the full database, without needing a full table scan.

The full field-by-field reference for every table, including every JSON attribute and taxonomy code, is in the Field Reference appendix of the User Manual.

`sources` (not shown above) records the origin dataset in more detail via its name and URI fields, such as the exact building dataset version used for a country.

3.2 Understanding number and relative_size

`entity_assets.number` is easy to misread, and it means something different depending on the category of the entity it is attached to:

- **Building entity (category = 1):** a specific, physical building has an uncertain classification, but it is still exactly one building.

The possible taxonomies for that one building are mutually exclusive outcomes, so their number values are probabilities that add up to 1 across the entity's asset pairs.

- **Tile entity (category = 0):** a tile aggregates many buildings. Here, a number is already an expected building count for that taxonomy, not a probability, as a tile can contain many buildings of a given type.

Summing numbers across a tile's asset pairs gives the tile's total expected building count, not 1.

In both cases, summing the number over a region gives you a genuine count of buildings of that type in that region. For a single building, the sum is capped at 1 (one building, distributed over its possible types), while for a tile, the sum reflects how many buildings of that type exist in it.

For example, a tile entity with two `entity_assets` rows, one with number 1.3 and one with number 0.8, represents roughly 2.1 buildings in total: 1.3 of the first taxonomy and 0.8 of the second.

`relative_size` is a separate, independent factor: it scales for the size of a specific asset's building relative to other buildings of a similar type. It does not change how number should be interpreted or summed for counting purposes. Still, it is used (together with number) when computing weighted totals of attributes such as population or structural cost. See the User Manual for worked examples using `exposurelib`.

This design lets the same `entity_assets` table serve both a single, specifically identified building and a whole tile of probable ones, without needing two separate schemas for the two cases.

4 Support and Contact Information

Before contacting support, please review the **FAQs** and **Additional resources** below.

This information will be provided once it becomes available.

4.1 Additional resources

Official documentation

- PostgreSQL: <https://www.postgresql.org/docs/>
- PostGIS: <https://postgis.net/documentation/>
- QGIS: <https://qgis.org/>

EU Building Database

- GDE getting started guide:
https://docs.dynamicexposure.org/gde/getting_started/ExploreData/

The Global Dynamic Exposure project website also publishes methodology background and release notes for the underlying model, which are useful background reading beyond the scope of this manual.

5 FAQs

- **Do I need PostGIS if I already have a PostgreSQL server running?**

Yes. PostGIS is not included by default — it is a separate package (`postgresql-17-postgis-3`) that must be installed and then enabled per database with `CREATE EXTENSION postgis;`. Having PostgreSQL already installed does not give you PostGIS.

- **Can I use a PostgreSQL version other than 17?**

This setup should also work with PostgreSQL 18, but version 17 is what this manual uses and has been written against. If you install a different version, make sure to install the matching PostGIS package for it (`postgresql-<version>-postgis-3`) — a PostGIS package built for a different server version will not load.

- **Do I have to create the separate consumer and contributor roles, or can I just use the postgres superuser for everything?**

You can use `postgres` for a quick local test, but it is not recommended beyond that. The `consumer/contributor` split lets you give most users and applications read-only access while keeping write access limited to the few that actually need it, which matters in particular once the database port is reachable from other machines (e.g. a QGIS workstation).

- **What happens if I run the schema import twice by accident?**

It will fail with "relation already exists" errors — the schema script creates tables with plain `CREATE TABLE` statements, which do not allow being run twice against the same database. If you need to start over, drop and recreate the database first.

- **Can I import a single country's data instead of the full EU dataset?**

Yes. The schema does not require a full EU-wide dataset to function; a dump scoped to one country imports and works the same way.

- **Is it safe to leave port 5432 open so a QGIS workstation can connect remotely?**

Only open it to the specific IP address(es) that need it, not to the internet at large. Leaving a database port open broadly is a common way databases get compromised.

- **Can multiple people query the database at the same time?**

Yes. PostgreSQL handles concurrent read connections without issue, and since exploration typically happens through the read-only `consumer` role, multiple people exploring data at once (via QGIS or `exposurelib`) will not conflict with each other or risk modifying the data.

- **Do I need to install `exposurelib` or `taxonomylib` to complete this installation?**

No. Those are only needed if you plan to query the database with Python afterwards, as covered in the User Manual. This manual's installation itself only requires PostgreSQL, PostGIS, and `psql`.

6 Glossary

Definitions of technical or application-specific terms in alphabetical order.

- **APT (Advanced Package Tool):** The package management system used on Ubuntu and other Debian-based Linux distributions. Used in this manual to install PostgreSQL and PostGIS from the official PostgreSQL repository.
- **Asset:** A possible building type (taxonomy, population, and cost attributes) that could apply to an entity, independently of any specific location. Stored in the assets table.
- **Building entity:** An entity representing one specific building footprint, as opposed to a tile entity's residual aggregate for buildings without one.
- **Database role:** A PostgreSQL entity that can own database objects and hold permissions. This manual defines a consumer role (read-only) and a contributor role (read-write), which inherit their privileges from the read_only and read_write roles, respectively.
- **Dump file:** A file produced by exporting the contents of a database (schema or data) so that it can be re-imported into another PostgreSQL instance. The schema dump and data dump referenced in this manual are examples of such files.
- **Entity:** A geographical feature in the model, either as a tile (category 0), a building (category 1), or a building part (category 2). Stored in the entities table.
- **Exposure database:** The PostgreSQL database (named exposure in this manual) that stores the EU Building Database exposure data.
- **Extension (PostgreSQL extension):** An add-on package that extends PostgreSQL's functionality. PostGIS is installed as an extension, enabled per database with the CREATE EXTENSION postgis; command.
- **GEM Taxonomy:** The building-taxonomy standard used to classify buildings by material, structural system, height, occupancy and other characteristics.
- **PostGIS:** A spatial database extension for PostgreSQL that adds support for geographic objects, allowing location-based queries and spatial data storage.
- **PostgreSQL:** An open-source object-relational database management system, used here to store and query the EU Building Database exposure data.
- **psql:** The command-line interactive client used to connect to and issue SQL commands against a PostgreSQL database.
- **QGIS:** An open-source geographic information system (GIS) application that can connect to a PostGIS-enabled PostgreSQL database to visualise and explore spatial layers.
- **Quadkey:** A Quadtree tile identifier (zoom level 18 in this model), used to identify tile entities and to record which tile each building entity falls within.
- **relative_size:** A field on entity_assets that scales an asset's contribution to weighted totals (such as population or cost) relative to other buildings of a similar type, independently of the number field.
- **Schema:** A namespace within a PostgreSQL database that groups related database objects, such as tables. This manual imports data into both the public schema and an exposure schema.

- **Superuser:** A PostgreSQL role with unrestricted access to the database, used in this manual via the postgres system account to perform installation and administration steps.
- **Taxonomy:** The set of key/value codes describing a building's construction type, stored as JSON in the taxonomies table and referenced by assets.taxonomy_id.
- **Tile entity:** An entity holding the residual, tile-level probabilistic aggregate for buildings expected within one Quadkey tile but not captured as their own building entities. It does not aggregate every building in the tile. Buildings with their own footprint are represented individually as building entities instead and are excluded from the tile's aggregate.